

티케팅마스터

TICKETING MASTER · 최종 프로젝트 기획안

대기열로 트래픽을 분산시키고, 낙관적 락으로 좌석 동시성을 보장하는 e 스포츠 티켓팅

① 프로젝트 개요

- 프로젝트명: 티켓팅마스터 (Ticketing Master)
- 한 줄 소개: 동시 접속 1 만 명 환경에서 대기열로 트래픽을 분산시키고, 그 뒤에서 DB 낙관적 락으로 좌석 동시성을 처리하는 e 스포츠 티켓 예매 플랫폼
- 도메인: e 스포츠 티켓팅 (LOL, VALORANT, OVERWATCH 등)
- 기준 시나리오: LCK 2026 스프링 개막전 T1 vs Gen.G, LoL Park 400 석
- 동시 접속: 약 1 만 명 (실측 1,000~3,000 VUser, 1 만은 추정)
- 대기열 입장 단위: 200 명씩 배치 처리
- 팀 / 기간 / 아키텍처: 4 명 풀스택 / 6 주 / 모놀리식

핵심 목표

- 좌석 동시성 제어로 중복 예약 0 건 달성
- 결제-좌석 차감 정합성 보장 (역등성 + 보상 트랜잭션)
- 대기열로 좌석 점유 단계의 동시 트래픽을 제어된 수준으로 분산
- 부하 테스트로 낙관적 락의 한계 지점을 수치로 확인 → 데이터 기반 의사결정

② 문제 정의: "서버는 왜 터지는가"

단순한 "수량 감소" 문제가 아닌, 실제 티켓팅 환경에서 발생하는 6 가지 장애를 카테고리로 분해한다

카테고리	발생 상황	본 프로젝트 대응
트래픽 폭주	오픈 시각 동시 접속 폭주, 커넥션 풀 고갈	✅ 대기열 200 명 단위 입장
DB 병목	동일 좌석 동시 조회/수정으로 Lock 경합	✅ 대기열 분산 + 인덱스
동시성	같은 좌석 동시 결제로 중복 예약	✅ DB 낙관적 락 (@Version)
재고 정합성	결제 도중 취소로 좌석 잠김	✅ DB 스케줄러로 점유 자동 해제
외부 연동	PG API 타임아웃, 네트워크 단절	🟡 역등성 키 + Mock PG 검증
장애 전파	결제 모듈 장애가 전체 확산	❌ 범위 제외

💡 본 프로젝트의 무게중심

위 6 개 중 [트래픽 폭주]와 [동시성]을 핵심으로 깊게 다룬다
 대기열이 앞단에서 트래픽을 1 만 → 200 명 단위로 분산시키고, 그 뒤에서 낙관적 락이 좌석
 단위 충돌을 처리하는 "두 단계 방어" 구조

③ 핵심 컨셉: 두 단계 방어

단일 기술이 아닌 "트래픽 특성에 맞는 단계별 흐름"으로 동시성에 접근한다

단계	역할	기술
1 단계	동시 입장 인원 제한	Redis Sorted Set + 200 명 단위 토큰
2 단계	좌석 단위 충돌 방지	JPA @Version (낙관적 락) + 재시도
방어선	락이 뚫려도 정합성 보장	BookingSeat UNIQUE 제약

좌석 상태 흐름

AVAILABLE → RESERVED → SOLD (또는 RESERVED → AVAILABLE 복구)

- RESERVED 진입 시 reserved_until(만료 시각, 점유 후 7 분) 기록
- RESERVED → AVAILABLE 복구 경로 두 가지
 - 본인 점유 해제: 즉시 복구
 - 스케줄러 만료: 30 초 주기로 reserved_until 지난 좌석 자동 복구
- 결제 완료 → SOLD 확정

④ 핵심 기능 × 기술 선택

4.1 대기열 입장 제어

- 문제: 1 만 명 동시 접속 시 백엔드/DB 커넥션 즉시 고갈, 좌석 점유 단계 충돌 폭증
- 선택: Redis Sorted Set (ZADD score=진입시각) + UUID 입장 토큰, 200 명 단위 ALLOWED 처리로 좌석 점유 동시 인원을 200 명 이하로 유지
- 검증: Spring Filter 에서 Queue-Token 헤더 검증, 토큰 없이 좌석 API 접근 불가
- 이력: 실시간 제어는 Redis, 최종 상태(WAITING/ALLOWED/EXPIRED)는 DB Queue 테이블에 기록

4.1.5 좌석 조회

- 실시간 갱신: 필요 시 프론트 폴링 방식으로 좌석 조회 API 를 주기적으로 호출 (현재는 미구현, 추후 필요 시 추가)

- 선택: DB Seat.status 를 좌석 상태의 기준값으로 사용, 2 단계 조회 흐름
 - 1 단계: GET /matches/{matchId}/sections — 구역 목록 + 구역별·등급별 잔여 좌석 수
 - 2 단계: GET /matches/{matchId}/sections/{sectionId}/seats — 구역 내 좌석 + 상태
- 잔여 카운트: status NOT IN (SOLD, RESERVED) 좌석만 카운트
- 최적화: 매치 좌석 한 번 로딩 + 메모리 집계 (N+1 방지)

4.2 좌석 점유 + 동시성 제어 (핵심)

- 문제: 동일 좌석 동시 점유로 중복 예약, 영구 점유 발생
- 선택: JPA 낙관적 락 (@Version), 충돌 시 OptimisticLockException + 1~2 회 재시도 (50~100ms)
- 점유 시 기록 필드:
 - Seat.status = RESERVED
 - Seat.reserved_until = now() + 7 분 ← TTL 명시
 - Seat.reserved_by = userId
 - Seat.version 자동 증가
- 방어선: BookingSeat (match_id, seat_id) UNIQUE 제약으로 DB 레벨 차단

왜 낙관적 락인가

낙관적 락이 가장 효율적 (락 자원을 점유하지 않아 DB 부하 최소)

대안 비교

방식	장점	단점 / 결정
비관적 락	안전, 단순	락 대기 누적 → 현재는 미적용
Redis 분산 락	분산 환경 대응	외부 의존성 추가 → 추후 필요 시 검토
낙관적 락 ★	충돌 적은 환경 최적	충돌 잦으면 재시도 누적 (대기열로 빈도 제어)

의사결정 분기

부하 테스트에서 인기 좌석 충돌이 누적되면, 그 좌석만 비관적 락 또는 Redis 락으로 이원화, 그렇지 않으면 낙관적 락 그대로 유지

4.3 결제 ↔ 좌석 차감 정합성

- 문제: 결제 성공인데 좌석 차감 실패 시 정합성 깨짐, 네트워크 재시도로 중복 결제 가능
- 선택: Booking(PENDING) → Payment 성공 → Booking(CONFIRMED) 단계 분리, 멍등성 키 (Idempotency Key)로 중복 결제 차단, 좌석 차감 실패 시 자동 환불 후 로그 기록
- 가격 스냅샷: BookingSeat.seat_price 로 예매 시점 가격 보존
- 결제 흐름: 토스페이먼츠 위젯 연동

- 프론트에서 “결제하기” 클릭 → 토스 위젯 오픈
- 사용자가 위젯 안에서 결제 수단(카드 / 간편결제 / 계좌이체) 선택
- 토스 응답으로 받은 paymentKey 를 백엔드에 전송 (POST /payments/confirm)
- 백엔드가 토스 승인 API 호출 후 Booking(CONFIRMED) 처리
- 카드 / 계좌이체 등 결제 수단 선택 UI 는 우리 프론트가 아닌 토스 위젯이 담당

4.4 좌석 점유 만료

- 선택: Spring @Scheduled 로 reserved_until 지난 RESERVED 좌석을 30 초 주기로 AVAILABLE 복구
- 왜 DB 스케줄러: 현재는 외부 의존성 최소화가 목표, Redis Keyspace Notification 은 추후 확장 옵션으로 보류

4.5 취소표 반환

- Booking.status = CANCELED → Seat.status = AVAILABLE 즉시 복구
- Match.cancel_available_until / Event.cancel_fee 기준으로 환불 금액 계산
- Payment 취소 후 PG 환불 호출, 실패 시 로그 기록
- 환불 처리 책임: 백엔드가 토스 환불 API 를 직접 호출, 테스트 환경에서도 동일하게 동작하며 실제 돈은 빠지지 않지만 토스 서버에서는 정상 환불 처리됨, Booking 에 cancelReason 필드를 두어 자동 취소(결제 시간 초과) / 수동 취소 사유를 기록

⑤ 차별화 포인트

- D1. 문제 정의의 구체성 — “왜 서버가 터지는가”를 6 개 카테고리로 분해, 1 만 동시접속/400 석/200 명 배치라는 구체적 시나리오
- D2. 기술 선택의 흐름 — 각 기술을 독립적으로 선택하지 않고, 앞 단계가 만든 환경을 전제로 다음 기술을 선택, 대기열로 동시 충돌을 낮춘 뒤, 저충돌 환경에 적합한 낙관적 락을 적용하는 단계적 설계
- D3. 적용 → 측정 → 결정의 반복 — 처음부터 완벽한 설계를 추구하지 않고, 단계별로 측정해 다음 결정을 내림, 부하 테스트 단계에서 낙관적 락의 실제 성능을 검증하고, 인기 좌석 충돌이 누적되면 마무리 단계에서 해당 좌석만 별도 락 전략으로 분리
- D4. 측정된 결과 — RPS/응답시간/에러율, 좌석 중복 예약 0 건, 낙관적 락 재시도 누적을 등 수치로 검증



선택 근거 요약

좌석 점유 - 낙관적 락 사용 (앞단 대기열로 동시 트래픽이 200 명 수준까지 줄어든 저충돌 환경, 락 자원을 점유하지 않아 DB 부하 최소화)

⑥ 데이터 모델 요약

엔티티	역할	비고
User	회원 / 관리자	role: USER / ADMIN
Event	대회 / 리그	메인 카드 단위, 예매 정책 보유
Match	회차 (개별 경기)	예매 최소 단위, 좌석/대기열 독립
Team	참가 팀	Match 에 home/away 연결
SeatGrade / Section	등급+가격 / 구역	Event 단위 공유
Seat	개별 좌석	Match 단위, @Version + reserved_until + reserved_by
Queue	대기열 엔트리	Redis 실시간 + DB 이력
Booking / BookingSeat	예매 + 좌석 매핑	(match_id, seat_id) UNIQUE, 가격 스냅샷
Payment	결제	Booking 과 1:1, PG 키 보유

핵심 설계 포인트

- Event = 대회, Match = 회차 → 메인 카드는 Event, 예매는 Match
- Seat 에 @Version 컬럼 → 낙관적 락의 핵심
- BookingSeat UNIQUE 제약 → 락이 뚫려도 중복 예약 차단
- BookingSeat.seat_price → 예매 시점 가격 보존
- 주요 인덱스: Seat(match_id, status), Queue(match_id, status), Booking(user_id, status)

⑦ 기술 스택 / 진행 전략

기술 스택

영역	선택
언어 / 프레임워크	Java 21, Spring Boot 3.x
DB / ORM	Oracle 23ai + Spring Data JPA + @Version
Cache / Queue	Redis 8 (Sorted Set 기반 대기열, 트래픽 분산 단계에서 본격 활용)
인증	JWT (Access: Authorization 헤더 / Refresh: Body, 쿠키 미사용)
프론트엔드	React + TypeScript, Zustand, shadcn/ui, TanStack Query
부하 테스트 / 인프라	k6 / Docker Compose / GitHub Actions

진행 단계

단계	기간	내용
1 단계 우선 돌아가게	Week 1~3	기본 기능 + 낙관적 락 좌석 점유 + DB 스케줄러 점유 만료, 대기열은 단순 폴링 또는 없음, Redis 는 세션 정도만
2 단계 트래픽 분산 + 한계 측정	Week 4~5	Redis Sorted Set 대기열 + 200 명 단위 토큰, k6 부하 테스트로 낙관적 락 한계 수치화
3 단계 의사결정 + 마무리	Week 6	결과에 따라 (a) 낙관적 락 유지 또는 (b) 인기 좌석만 이원화 발표 준비, 문서화

⑧ 검증 방법

부하 테스트 시나리오 (k6)

- 1 만 명 동시 접속 → 메인/상세 페이지 응답시간 측정
- 200 명 동시 좌석 점유 시도 → 인기 좌석 충돌 빈도 측정
- 1 만 명 대기열 진입 → 200 명 단위 입장 처리 시간 측정

측정 지표

- RPS, 평균/p95/p99 응답시간, 에러율
- 좌석 중복 예약 발생 건수 (반드시 0)
- OptimisticLockException 발생률, 재시도 성공률
- DB Connection 사용률, Redis 메모리 사용량

프로젝트 한 줄 정의

트래픽 폭주는 대기열로 분산시키고, 좌석 동시성은 그 뒤에서 낙관적 락으로 처리하는, 두 단계 방어 구조의 e 스포츠 티켓 예매 시스템